

Inteligencia Artificial y Neurociencias

Licenciatura en Tecnología Digital, UTDT, 2S-2026

Profesores: Agustín Gravano, Andrés Rieznik

Auxiliares: Gerardo Del Toro, Antonella Giordano Furchi

Métodos de Diferencias Temporales: SARSA y Q-learning

Repaso: Algoritmo Monte Carlo para estimar π_*

1. Generar un episodio: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3$
 - A_t son las acciones elegidas por el agente según π .
 - R_t, S_t son los efectos determinados por el ambiente según $p(s', r|s, a)$.
 - π es una política ε -greedy basada en $Q(s, a)$. No siempre es así, puede ser cualquier otra política
 - $Q(s, a)$ es una estimación de $q_\pi(s, a)$: el retorno esperado de ejecutar a en s y luego seguir la política π .
2. Al final del episodio, actualizar:
 - $Q(S_0, A_0)$, incorporando $G_0 = R_1 + \gamma R_2 + \gamma^2 R_3$;
 - $Q(S_1, A_1)$, incorporando $G_1 = R_2 + \gamma R_3$;
 - $Q(S_2, A_2)$, incorporando $G_2 = R_3$;
 - π , según los nuevos valores de Q .
3. Repetir

Repaso: Algoritmo Monte Carlo para estimar π_*

Después de k episodios que ejecutaron la acción a en el estado s , el valor estimado de (s, a) es: **Esto se hace para todo $Q(S_i, A_i) = (G_i(1) + \dots)/k$**

$$Q = \frac{G^{(1)} + G^{(2)} + G^{(3)} + \dots + G^{(k)}}{k}$$

donde $G^{(i)}$ es el retorno observado desde (s, a) en la i -ésima iteración.

Al terminar el episodio $k + 1$, el valor estimado de (s, a) es:

$$\begin{aligned} Q' &= \frac{G^{(1)} + G^{(2)} + G^{(3)} + \dots + G^{(k)} + G^{(k+1)}}{k + 1} \\ &= \frac{Q \cdot k + G^{(k+1)}}{k + 1} \\ &= \frac{Q \cdot k}{k + 1} + \frac{G^{(k+1)}}{k + 1} + \frac{Q}{k + 1} - \frac{Q}{k + 1} \\ &= Q \cdot \frac{k + 1}{k + 1} + \frac{G^{(k+1)}}{k + 1} - \frac{Q}{k + 1} \\ &= Q + \frac{1}{k + 1} \cdot (G^{(k+1)} - Q) \end{aligned}$$

Métodos de Diferencias Temporales (TD)

La generación de episodios son siempre secuenciales

Recordemos la asignación que vimos hace un tiempo:

$$NuevaEst \leftarrow ViejaEst + TasaApr \left[Objetivo - ViejaEst \right]$$

Los **métodos MC** actualizan la estimación de $q_\pi(s, a)$ al final de cada episodio, una vez conocida una medición real del objetivo G_t :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[G_t - Q(S_t, A_t) \right]$$

Los **métodos TD**, en cambio, **no** esperan hasta el final del episodio. Actualizan Q luego de tomar cada acción A_t y observar (S_{t+1}, R_{t+1}) :
Actualizan en cada paso

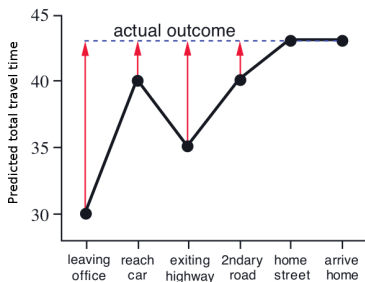
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

donde A_{t+1} es una acción desde S_{t+1} elegida según una política basada en Q (p.ej., ϵ -greedy). En lugar de medir G_t , TD basa su estimación en otro valor estimado (a esto se lo llama *bootstrapping*¹).

¹ Sin relación con el método estadístico de muestreo con reemplazo.

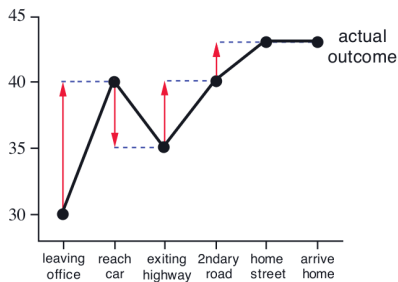
MC vs. TD - Ejemplo: Conduciendo a casa

Estado	Hora	Tiempo restante predicho	Tiempo total predicho
Saliendo del trabajo	18:00	30	30
En el auto, llueve	18:05	35	40
Saliendo de autopista	18:20	15	35
Ruta local, atrás de camión	18:30	10	40
Calle de casa	18:40	3	43
Entrando en casa	18:43	0	43



Métodos Monte Carlo

MC espera a llegar a casa, recién ahí sabe que el total real fue 43, y corrige todas las predicciones anteriores hacia ese 43



Métodos TD

Cada flecha apunta a la predicción del paso siguiente, no al final. Va aprendiendo sobre la marcha.

Estimación TD on-policy de π_* : Algoritmo SARSA

Inicializar $Q(s, a)$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Repetir:

 Ejecutar la acción A ; observar R, S'

$A' \leftarrow$ acción desde S' según política ε -greedy basada en Q

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'$

$A \leftarrow A'$

 hasta que S sea terminal

La tasa de aprendizaje $\alpha \in (0, 1]$ es un parámetro permite controlar la relevancia de la información más reciente, lo cual es útil en ambientes no estacionarios.

A es la acción que realmente vas a tomar en S, elegida con la misma política ε -greedy

Estimación TD off-policy de π_* : Q-learning

SARSA usa la misma política ε -greedy basada en Q para elegir la siguiente acción A_{t+1} y para estimar el retorno G_t (en rojo) en la regla de actualización de Q :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

Por eso, SARSA es un método on-policy.

Con el siguiente cambio a la regla de actualización de Q :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

desacoplamos la estimación del retorno G_t (ahora se supone una política greedy pura) de la política seguida para elegir la siguiente acción del episodio (ε -greedy).

Por eso, Q-learning es un método off-policy.

Asume una política greedy pura para estimar el retorno,
sin importar qué acción vayas a tomar de verdad después.

Estimación TD off-policy de π_* : Q-learning

Inicializar $Q(s, a)$ arbitrariamente $\forall s \in \mathcal{S}, a \in \mathcal{A}(s)$

Repetir:

 Inicializar S

 Repetir:

$A \leftarrow$ acción desde S según política ε -greedy basada en Q

 Ejecutar la acción A ; observar R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 hasta que S sea terminal

La tasa de aprendizaje $\alpha \in (0, 1]$ es un parámetro permite controlar la relevancia de la información más reciente, lo cual es útil en ambientes no estacionarios.